

ProjectManagerApp — Hướng Dẫn 02: Data Layer

Package: `com.projectmanager.config` · `com.projectmanager.repository`

Sơ Đồ Data Layer

```

ProjectService / AuthService
    ↓ gọi
TaskRepository / UserRepository    ← toàn bộ SQL đặt tại đây (DAO Pattern)
    ↓ dùng
DatabaseConfig.getConnection()      ← Singleton – 1 connection duy nhất
    ↓ kết nối
SQL Server – ProjectManagerDB (Tasks + Users)
  
```

DatabaseConfig.java — Singleton Pattern

```

package com.projectmanager.config;

import java.sql.Connection;
import java.sql.DriverManager;

// 
// | SINGLETON PATTERN – 3 dấu hiệu nhận biết: |
// | 1. private static field lưu instance duy nhất |
// | 2. private constructor – không ai new DatabaseConfig() |
// | 3. public static getConnection() – điểm truy cập duy nhất |
// 
public class DatabaseConfig {

    private static Connection connection; // 1. Instance duy nhất

    private static final String URL      =
"jdbc:sqlserver://localhost:1433;databaseName=ProjectManagerDB;encrypt=false";
    private static final String USER     = "sa";
    private static final String PASSWORD = "your_password";

    private DatabaseConfig() {} // 2. Private constructor

    // 3. Điểm truy cập duy nhất – synchronized tránh race condition multi-thread
    public static synchronized Connection getConnection() {
        try {
            if (connection == null || connection.isClosed()) {
                connection = DriverManager.getConnection(URL, USER, PASSWORD);
            }
        } catch (Exception e) {
            System.out.println("[DB] Lỗi kết nối: " + e.getMessage());
        }
    }
  
```

```
    }  
    return connection;  
}  
}
```

TaskRepository.java — DAO Pattern

```
package com.projectmanager.repository;  
  
import com.projectmanager.config.DatabaseConfig;  
import com.projectmanager.models.*;  
import java.sql.*;  
import java.util.*;  
  
// DAO Pattern: tất cả SQL của Tasks tập trung tại đây  
// ProjectService và các Controller KHÔNG viết SQL – chỉ gọi method của repository  
public class TaskRepository {  
  
    private final Connection conn = DatabaseConfig.getConnection();  
  
    // — INSERT —  
  
    public boolean insert(Task t) {  
        String sql = "INSERT INTO Tasks "  
            + "  
(TaskId,Title,Priority,Status,TaskType,Severity,EstimatedHours,AssignedTo) "  
            + "VALUES (?, ?, ?, ?, ?, ?, ?, ?)";  
        try (PreparedStatement ps = conn.prepareStatement(sql)) {  
            ps.setString(1, t.id);  
            ps.setString(2, t.title);  
            ps.setString(3, t.priority);  
            ps.setString(4, t.status);  
            ps.setString(5, t.getTypeCode()); // "B" hoặc "F"  
            if (t instanceof Bug) {  
                Bug b = (Bug) t;  
                ps.setString(6, b.severity);  
                ps.setNull(7, Types.INTEGER);  
                ps.setNull(8, Types.NVARCHAR);  
            } else {  
                Feature f = (Feature) t;  
                ps.setNull(6, Types.VARCHAR);  
                ps.setInt(7, f.estimatedHours);  
                if (f.isAssigned()) ps.setString(8, f.getAssignedTo());  
                else ps.setNull(8, Types.NVARCHAR);  
            }  
            ps.executeUpdate();  
            return true;  
        } catch (SQLException e) {  
            System.out.println("[REPO] Insert loi: " + e.getMessage());  
            return false;  
        }  
    }  
}
```

```

    }
}

// — SELECT ALL —————

// Sắp xếp: HIGH priority trước, cùng priority theo TaskId
public List<Task> findAll() {
    List<Task> list = new ArrayList<>();
    String sql = "SELECT * FROM Tasks ORDER BY "
        + "CASE Priority WHEN 'HIGH' THEN 1 WHEN 'MEDIUM' THEN 2 ELSE 3
END, TaskId";
    try (Statement st = conn.createStatement()) {
        ResultSet rs = st.executeQuery(sql);
        while (rs.next()) list.add(mapRow(rs));
    } catch (SQLException e) { e.printStackTrace(); }
    return list;
}

// — SELECT BY ID —————

public Task findById(String id) {
    String sql = "SELECT * FROM Tasks WHERE TaskId = ?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) return mapRow(rs);
    } catch (SQLException e) { e.printStackTrace(); }
    return null;
}

// — UPDATE STATUS —————

// Chỉ update cột Status – dùng khi user đổi trạng thái task trong
TaskListController
public boolean updateStatus(String id, String newStatus) {
    String sql = "UPDATE Tasks SET Status = ? WHERE TaskId = ?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, newStatus);
        ps.setString(2, id);
        return ps.executeUpdate() > 0; // > 0 = ít nhất 1 row bị ảnh hưởng
    } catch (SQLException e) { e.printStackTrace(); return false; }
}

// — DELETE —————

// Admin only – TaskListController kiểm tra UserSession.isAdmin() trước khi
gọi
public boolean delete(String id) {
    String sql = "DELETE FROM Tasks WHERE TaskId = ?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, id);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) { e.printStackTrace(); return false; }
}

```

```

// — mapRow – discriminator pattern —————
// Đọc TaskType column → tái tạo đúng subclass Bug hoặc Feature
private Task mapRow(ResultSet rs) throws SQLException {
    String type = rs.getString("TaskType"); // "B" hoặc "F"
    if ("B".equals(type)) {
        Bug b = new Bug();
        b.id = rs.getString("TaskId");
        b.title = rs.getString("Title");
        b.priority = rs.getString("Priority");
        b.status = rs.getString("Status");
        b.severity = rs.getString("Severity");
        return b;
    } else {
        Feature f = new Feature();
        f.id = rs.getString("TaskId");
        f.title = rs.getString("Title");
        f.priority = rs.getString("Priority");
        f.status = rs.getString("Status");
        f.estimatedHours = rs.getInt("EstimatedHours");
        f.assign(rs.getString("AssignedTo")); // null-safe qua IAssignable
        return f;
    }
}
}

```

UserRepository.java

```

package com.projectmanager.repository;

import com.projectmanager.config.DatabaseConfig;
import com.projectmanager.models.entity.User;
import java.sql.*;
import java.util.*;

// DAO Pattern: tất cả SQL của Users tập trung tại đây
// AuthService dùng để login/register; UserListController (admin) dùng
// findAll/updateStatus
public class UserRepository {

    private final Connection conn = DatabaseConfig.getConnection();

    // — Tìm theo username – dùng khi login —————

    public User findByUsername(String username) {
        String sql = "SELECT * FROM Users WHERE Username = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, username);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return mapRow(rs);
        }
    }
}

```

```

        } catch (SQLException e) { e.printStackTrace(); }
        return null;
    }

    public boolean existsByUsername(String username) {
        return findByUsername(username) != null;
    }

    // — Lấy tất cả users – admin only (UserController) —————

    public List<User> findAll() {
        List<User> list = new ArrayList<>();
        String sql = "SELECT * FROM Users ORDER BY Role DESC, Username"; // admin
trước
        try (Statement st = conn.createStatement()) {
            ResultSet rs = st.executeQuery(sql);
            while (rs.next()) list.add(mapRow(rs));
        } catch (SQLException e) { e.printStackTrace(); }
        return list;
    }

    // — INSERT – dùng khi register —————

    public boolean insert(User u) {
        String sql = "INSERT INTO Users(Username,PasswordHash,Email,Role,Status)
VALUES(?,?,?,?,?)";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, u.username);
            ps.setString(2, u.passwordHash);
            ps.setString(3, u.email == null ? "" : u.email);
            ps.setString(4, u.role);
            ps.setBoolean(5, u.status);
            ps.executeUpdate();
            return true;
        } catch (SQLException e) {
            System.out.println("[REPO] Insert user loi: " + e.getMessage());
            return false;
        }
    }

    // — Block/Unblock user – admin only (UserController) —————

    // Toggle: status=true → active, status=false → blocked
    public boolean updateStatus(int userId, boolean newStatus) {
        String sql = "UPDATE Users SET Status = ? WHERE UserId = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setBoolean(1, newStatus);
            ps.setInt(2, userId);
            return ps.executeUpdate() > 0;
        } catch (SQLException e) { e.printStackTrace(); return false; }
    }

    // — mapRow —————

```

```

private User mapRow(ResultSet rs) throws SQLException {
    User u = new User();
    u.id = rs.getInt("UserId");
    u.username = rs.getString("Username");
    u.passwordHash = rs.getString("PasswordHash");
    u.email = rs.getString("Email");
    u.role = rs.getString("Role");
    u.status = rs.getBoolean("Status");
    return u;
}
}

```

Ghi Chú Thiết Kế Data Layer

Điểm	Giải thích
<code>PreparedStatement</code> thay vì <code>Statement</code>	Chống SQL Injection — ? được escape tự động
<code>try-with-resources (ps)</code>	<code>PreparedStatement</code> tự <code>close()</code> — tránh resource leak
<code>mapRow(ResultSet)</code> trong <code>TaskRepo</code>	Discriminator: <code>TaskType='B' → new Bug()</code> , <code>'F' → new Feature()</code>
<code>updateStatus()</code> trong cả 2 repo	<code>TaskListController</code> đổi task status; <code>UserListController</code> block/unblock user
<code>UserRepository.findAll()</code>	Chỉ gọi từ <code>UserListController</code> — controller đã kiểm tra <code>UserSession.isAdmin()</code> trước
<code>setNull(idx, Types.XXX)</code>	Rõ ràng set NULL cho cột không thuộc loại task đó — tránh lỗi schema constraint

Discriminator Pattern — INSERT vs SELECT

Khi INSERT Bug (TaskType='B'):

```

Severity      = "HIGH"      ← set
EstimatedHours = NULL       ← Bug không có
AssignedTo    = NULL        ← Bug không có

```

Khi INSERT Feature (TaskType='F'):

```

Severity      = NULL        ← Feature không có
EstimatedHours = 20         ← set
AssignedTo    = "Nguyen A"  ← set nếu có, NULL nếu chưa assign

```

Khi SELECT → `mapRow()`:

```

TaskType = 'B' → new Bug(),      set severity
TaskType = 'F' → new Feature(),  set estimatedHours + assign()

```